System Design Masterclass Vol. 4: High-Level Design (HLD) — Real-World Enterprise Architectures and Interview Execution Framework

Abstract

The capstone engineering masterclass of the 30-day intensive System Design curriculum, designed for Days 23–30. This volume synthesizes distributed systems primitives into battle-tested, end-to-end architectures capable of handling real-world global scale. It arms engineers with the proven 45-minute High-Level Design (HLD) interview framework, covering structured requirements discovery, back-of-the-envelope capacity estimations (QPS, storage, network bandwidth), microservice decomposition, and deep-dive bottleneck resolution. It provides comprehensive architectural blueprints for four iconic distributed systems: (1) Scalable URL Shortener (TinyURL) handling 100M daily writes with Base62 encoding and pre-generated key ranges; (2) Global Real-Time Chat System (WhatsApp) utilizing WebSockets, connection managers, presence servers, and offline storage; (3) Proximity-Based Ride-Sharing Architecture (Uber) powered by Geohash for 2D geospatial indexing and sub-second driver matching; and (4) High-Concurrency Flash Sale and E-Commerce Checkout (Amazon/Flipkart) featuring distributed inventory locking via atomic Redis Lua scripts, the Saga pattern for distributed transactions, and idempotency guarantees.

Index Terms

binary trees, algorithms, data structures, computational complexity, tree traversal, recursive algorithms, formal analysis

1 Introduction: Real-World Architecture & Interview Mastery

Week 4 (Days 23 to 30) is the capstone of the 30-day System Design curriculum. Having mastered Low-Level Design (Volumes 1 & 2) and Distributed Primitives (Volume 3), you will now synthesize these components into complete, end-to-end architectures that solve real-world industrial problems.

In top-tier engineering interviews (Google, Meta, Amazon, Uber, Flipkart), you are evaluated not on reciting component definitions, but on your ability to:

1) Drive a structured 45-minute architectural conversation.
2) Estimate capacity (QPS, storage, bandwidth) rapidly from first principles.
3) Navigate trade-offs between consistency, availability, latency, and cost.
4) Propose battle-tested defenses against single points of failure, hotspots, and data corruption.

This volume provides the 45-Minute HLD Interview Execution Framework followed by four comprehensive, production-grade case studies: TinyURL, WhatsApp, Uber, and the Amazon/Flipkart Flash Sale Engine.

2 The 45-Minute HLD Interview Execution Framework

An HLD interview is an open-ended collaborative design session. Successful candidates follow a strict 5-phase time management blueprint.

TABLE 1
The 45-Minute High-Level Design Interview Time Blueprint

Time	Phase	Candidate Goals & Actions
0 – 5 mins	Scope & Requirements	Clarify Functional (core user actions) and Non-Functional (QPS, latency, SLA) requirements.
5 – 10 mins	Capacity Estimations	Back-of-the-envelope calculations: QPS (Read/Write), Daily Storage, Bandwidth, Memory.
10 – 25 mins	High-Level Architecture	Draw end-to-end diagram: Client → CDN/DNS → LB → App Services → DB/Cache/Queue.
25 – 40 mins	Deep Dive & Bottlenecks	Address bottlenecks: DB sharding keys, cache eviction, concurrency locks, failure recovery.
40 – 45 mins	Wrap-up & Trade-offs	Summarize key trade-offs, monitoring/metrics, and future extensibility.

2.1 Back-of-the-Envelope Math Cheat-Sheet

- 1 Day = 86,400 seconds $\approx 100,000$ seconds (for rapid mental math).
- Average QPS = $\frac{\text{Daily Requests}}{100{,}000}$.
- Peak QPS = $2\times$ to $5\times$ Average QPS.
- Storage: 1 Byte = 8 bits; 1 KB = 10^3 bytes; 1 MB = 10^6 bytes; 1 GB = 10^9 bytes; 1 TB = 10^{12} bytes; 1 PB = 10^{15} bytes.

3 Case Study 1: Scalable URL Shortener (TinyURL)

3.1 1. Requirements & Capacity Estimation

- Functional: Given a long URL, generate a shorter alias (e.g., https://tiny.url/aB3x9z1). Redirecting to short URL navigates to original destination. Custom aliases supported.
- Non-Functional: High availability (99.99%), low-latency redirection (< 20ms), non-predictable URLs.
- Scale: 100 Million new URLs created per day. Read-to-Write ratio = 10:1 (1 Billion redirections/day).
  - Write QPS: $100\text{M}/100,000 = 1,000$ writes/sec (Peak: 2,000/sec).
  - Read QPS: $1\text{B}/100,000 = 10,000$ reads/sec (Peak: 20,000/sec).
  - Storage per URL: ≈ 500 bytes. Daily storage = $100\text{M}\times 500\text{B} = 50$ GB/day $\implies$ 18.25 TB/year.

3.2 2. Short URL Encoding: Base62 vs. Hashing

- Base62 alphabet: $[a-z, A-Z, 0-9]$ (62 characters).
- A 7-character Base62 string yields $62^7 \approx 3.52$ Trillion unique combinations, sufficient for decades.
- The Key Generation Service (KGS) Pattern: Rather than hashing long URLs on the fly (which causes hash collisions), use a standalone Key Generation Service (KGS). KGS pre-generates 7-character Base62 keys in batches and stores them in memory/DB. When an app server needs a key, KGS assigns an unused key in $O(1)$ without locking or collision checks.

3.3 3. Redirection Status Code: 301 vs. 302

- HTTP 301 (Moved Permanently): Browser caches redirection locally. Subsequent clicks bypass your servers completely. Saves server bandwidth, but prevents click analytics tracking.
- HTTP 302 (Found / Temporary): Browser asks TinyURL server every time. Enables click counting, geographic analytics, and link expiration.

3.4 4. Storage & Caching Layer

- Database: NoSQL Key-Value store (DynamoDB or Cassandra) with partition key = short_url.
- Caching: Redis cluster storing top 20% hottest URLs (80-20 Pareto Rule). 20% of 1B reads/day $\times$500 bytes $\approx$ 100 GB RAM.

4 Case Study 2: Real-Time Chat System (WhatsApp / Slack)

4.1 1. Requirements & Architecture

- 500 Million Daily Active Users (DAU).
- Low latency (< 100ms) 1-on-1 and small group messaging.
- Delivery receipts: Sent, Delivered, Read.
- Offline message storage and User Presence (Online / Last Seen).

4.2 2. Communication Protocol: WebSockets

HTTP is stateless and client-initiated (inefficient polling). Chat systems establish persistent, bidirectional, full-duplex WebSocket connections over TCP between client devices and backend Chat Gateway servers.

4.3 3. End-to-End Message Flow

1) Connection Phase: User A opens app; establishes WebSocket connection with Chat Server 1. Chat Server registers User A's active session in a Redis Session Store (userA -> server_1_ip).
2) Message Send Phase: User A sends message for User B to Chat Server 1.
3) Routing Phase: Chat Server 1 queries Session Store for User B.
   - Case 1: User B is Online on Server 2: Chat Server 1 publishes message to a Kafka topic or internal message bus targeting Server 2. Server 2 pushes message down User B's active WebSocket connection.
   - Case 2: User B is Offline: Message is saved to persistent storage (Apache Cassandra) and an Apple Push Notification (APNs) / Firebase Cloud Messaging (FCM) push notification is triggered.
4) Receipts: When User B's client confirms receipt, a lightweight ACK packet travels back, updating status to "Delivered" (double checkmarks).

4.4 4. Presence Service: Heartbeat Management

Tracking online status for 500M users via active pings overwhelms servers. Instead, clients emit a lightweight heartbeat every 30 seconds. The Presence Service sets a Redis key presence:userId with a 45-second TTL. If the client disconnects or loses signal, the key naturally expires without expensive cleanup routines.

5 Case Study 3: Proximity-Based Ride-Sharing (Uber / Lyft)

5.1 1. The Core Engineering Challenge

Hundreds of thousands of active drivers broadcast their GPS coordinates every 4 seconds. When a rider requests a pickup, the system must search the 2D spatial coordinate space and identify the top 5 nearest eligible drivers within a 3km radius in under 1 second.

5.2 2. Why Relational SQL Fails

Querying a standard SQL table with indices on ‘latitude‘ and ‘longitude‘:

SELECT * FROM drivers
WHERE lat BETWEEN 12.90 AND 12.95 AND lon BETWEEN 77.55 AND 77.60;

requires a 2D composite range scan. As driver updates flood the database, index lock contention and full-table scans bring response times to several seconds.

5.3 3. Spatial Partitioning: Geohash & Google S2

- Geohash: Recursively subdivides the earth into a hierarchical grid of bounding boxes, encoding latitude and longitude into an alphanumeric string (e.g., tdr1v7).
- Prefix Invariant: Points that share a long common Geohash prefix are physically close. A 6-character Geohash defines a bounding box of roughly 1.2 km $\times$ 0.6 km.
- Searching Neighbors: To locate nearby drivers, query the rider's Geohash plus the 8 adjacent surrounding Geohash cells to eliminate boundary edge effects.

5.4 4. System Architecture

- Location Ingestion Service: 500,000 drivers broadcasting every 4 seconds = 125,000 location writes/sec. Handled by an in-memory Redis cluster partitioned by Geohash.
- Dispatch & Matching Service: Queries Redis for candidate drivers in the 9-cell Geohash cluster, evaluates driver ETA using road network routing, and emits dispatch offers via WebSockets.

6 Case Study 4: High-Concurrency Flash Sale (Amazon / Flipkart)

6.1 1. The High-Scale Concurrency Problem

Consider a flash sale: 10,000 units of a limited-edition smartphone go on sale at 12:00:00 PM. Over 1,000,000 users click "Buy Now" in the opening 5 seconds. If 100,000 concurrent requests attempt to execute:

UPDATE inventory SET stock = stock - 1 WHERE item_id = 101 AND stock > 0;

the database encounters severe row-level lock contention, connection pool exhaustion, and catastrophic failure. Worse, uncoordinated distributed systems suffer from over-selling (selling 10,500 units when only 10,000 existed).

6.2 2. Production Architectural Defenses

1) Rate Limiting & Virtual Queue (Waiting Room): Drop 90% of duplicate bot clicks at the CDN and API Gateway layer using Token Bucket rate limiting. Direct excess human traffic to a Virtual Waiting Room.
2) In-Memory Inventory Reservation (Redis Lua Scripts): Database disk I/O cannot handle 200,000 QPS. Pre-warm Redis with stock: item_101_stock = 10000. When a user requests purchase, execute an atomic Lua script inside Redis:

```
-- Atomic Redis Lua script for inventory decrement
local stock = tonumber(redis.call('get', KEYS[1]))
if stock > 0 then
   redis.call('decr', KEYS[1])
   return 1 -- Success
else
   return 0 -- Sold out!
end
```

Redis executes Lua scripts in a single-threaded event loop, guaranteeing 100% atomicity with zero database lock contention!

3) Asynchronous Order Processing (Kafka): Users who successfully reserve a Redis stock token receive an immediate "Order Reserved" status. Their order payload is pushed into an Apache Kafka topic. Downstream checkout workers process orders, interact with payment gateways, and persist records to the primary database asynchronously.
4) Idempotency Keys: Network timeouts cause clients to retry checkout requests. Every purchase request carries a client-generated UUID Idempotency-Key. The backend stores this key in Redis for 24 hours, ensuring duplicate clicks never trigger duplicate charges or double deductions.
5) Compensating Transactions (Saga Pattern): If payment fails or times out after 10 minutes, a compensating transaction automatically increments the Redis inventory counter: redis.call('incr', KEYS[1]), releasing the stock back to the sale pool.

7 Daily 2-Hour Action Plan for Days 23 to 30

- Day 23 (2 hrs): Practice the 45-minute HLD framework. Memorize back-of-the-envelope capacity formulas (QPS, storage, bandwidth).
- Day 24 (2 hrs): Design TinyURL on a whiteboard or paper. Detail the Base62 encoding, Key Generation Service (KGS), and caching strategy.
- Day 25 (2 hrs): Design a Real-Time Chat System (WhatsApp). Map the WebSocket connection lifecycle, Session Store, and offline message storage.
- Day 26 (2 hrs): Design Uber / Ride-Sharing. Contrast standard SQL indexing against Geohash and QuadTrees for sub-second spatial matching.
- Day 27 (2 hrs): Design a Flash Sale / E-Commerce Checkout Engine. Walk through atomic Redis Lua reservations, Kafka decoupling, and idempotency keys.
- Day 28 (2 hrs): Mock Interview 1: Complete a timed 45-minute design of Netflix / YouTube Video Streaming (Chunking, Transcoding, CDN).
- Day 29 (2 hrs): Mock Interview 2: Complete a timed 45-minute design of a Distributed Rate Limiter (Redis Sliding Window).
- Day 30 (2 hrs): Final Curriculum Review: Review the LLD cheat-sheet (Volume 1 & 2) and HLD distributed primitives (Volume 3). You are ready!